Міністерство освіти і науки України

Фаховий коледж зварювання та електроніки імені Є.О. Патона

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №7

навчальної дисципліни

ОПЕРАЦІЙНІ СИСТЕМИ

ФКЗЕ. F2ОС07. 07ЛР

Шифр роботи

Студент Дєтков Н.

Група ПЗ-25-1/9

Перевірив Костенко А.О.

2026 - 2027

Тема: Вивчення файлової системи та функцій по обробці та управлінню даними в ОС Linux.

Мета: навчитися працювати з файловою системою Linux засобами командного рядка: орієнтуватися в структурі FHS, виконувати повний цикл файлових операцій, шукати файли за різними критеріями, розрізняти жорсткі й символічні посилання, монтувати та відмонтовувати носії, аналізувати використання дискового простору й архівувати дані.

Обладнання: ПК або віртуальна машина з Ubuntu 22.04; термінал; право виконувати sudo; змінний носій або додатковий віртуальний диск.

Варіант: 7.

Хід роботи

1 Отримав у викладача номер індивідуального варіанта 7. За таблицею варіантів мені дістався такий набір: вихідний каталог — /usr/include; критерій пошуку за розміром — файли .h, більші за 10 КБ; формат архіву — zip; додаткове завдання — пояснити різницю між du і df для того самого каталогу. Роботу виконував на віртуальній машині VirtualBox з Ubuntu 24.04 LTS під обліковим записом nikita (член групи sudo), у терміналі bash. Усі дії, що створюють і видаляють файли, виконував лише всередині навчального каталогу ~/lab7, а команди mount і umount — лише на змінному носії (флешка, яку видав викладач).

2 Створив структуру каталогів і підготував тестові дані. Командою mkdir -p ~/lab7/{src,docs,bak,mnt} створив одразу чотири підкаталоги (фігурні дужки розгортає сама оболонка, а ключ -p створює й батьківський каталог, якщо його ще немає). Перейшов у ~/lab7 командою cd і скопіював у src три файли з каталогу варіанта: stdio.h, stdlib.h і string.h (усі три мають розмір понад 10 КБ, тож пасують до критерію пошуку). Вміст src переглянув командою ls -lh (ключ -l виводить докладний формат, -h — розміри у зручних одиницях), а структуру — командою tree -L 1 (ключ -L 1 обмежує глибину одним рівнем). Утиліта tree не входить до

типового складу Ubuntu, тому довелося встановити її командою sudo apt install -y tree zip unzip (zip і unzip знадобляться в пункті 9).

```
nikita@ubuntu-vm:~$ sudo apt install -y tree zip unzip
nikita@ubuntu-vm:~$ which tree zip unzip
/usr/bin/tree
/usr/bin/zip
/usr/bin/unzip

nikita@ubuntu-vm:~$ mkdir -p ~/lab7/{src,docs,bak,mnt}
nikita@ubuntu-vm:~$ cd ~/lab7
nikita@ubuntu-vm:~/lab7$ cp /usr/include/stdio.h /usr/include/stdlib.h
   /usr/include/string.h src/
nikita@ubuntu-vm:~/lab7$ ls -lh src/
total 88
-rw-r--r-- 1 nikita nikita  29K Oct 27 14:08 stdio.h
-rw-r--r-- 1 nikita nikita  33K Oct 27 14:08 stdlib.h
-rw-r--r-- 1 nikita nikita  17K Oct 27 14:08 string.h

nikita@ubuntu-vm:~/lab7$ tree -L 1
.
├── bak
├── docs
├── mnt
└── src

4 directories, 0 files
```

Лістинг 1 - Створення структури каталогів і копіювання файлів із /usr/include

3 Вивів метадані файлу src/stdio.h командою stat і визначив реальний тип усіх трьох файлів командою file src/*. У виводі stat поле Inode (1314871) — номер індексного дескриптора, службового запису файлової системи, у якому зберігаються розмір, власник, права, часові мітки й адреси блоків із даними; ім’я файлу в inode не зберігається, його містить запис у каталозі. Поле Links (1) показує, скільки імен (жорстких посилань) вказує на цей inode; поки лічильник не дорівнює нулю, дані не видаляються. Поле Access містить права доступу в двох виглядах: вісімковому 0644 і символьному -rw-r--r-- (власник читає й пише, група та решта лише читають), а також власника (Uid) і групу (Gid). Команда file визначила всі три файли як вихідний код мовою C у текстовому ASCII-форматі: тип вона визначає за вмістом, а не за розширенням .h. Для порівняння перевірив виконуваний файл /bin/ls: це ELF-програма, хоча розширення в неї немає.

```
nikita@ubuntu-vm:~/lab7$ stat src/stdio.h
  File: src/stdio.h
  Size: 29037       Blocks: 64         IO Block: 4096   regular file
Device: 8,2 Inode: 1314871     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  nikita)   Gid: ( 1000/  nikita)
Access: 2026-10-27 14:08:11.236884021 +0200
Modify: 2026-10-27 14:08:11.236884021 +0200
Change: 2026-10-27 14:08:11.236884021 +0200
 Birth: 2026-10-27 14:08:11.236884021 +0200

nikita@ubuntu-vm:~/lab7$ file src/*
src/stdio.h:  C source, ASCII text
src/stdlib.h: C source, ASCII text
src/string.h: C source, ASCII text
```

```
nikita@ubuntu-vm:~/lab7$ file -b /bin/ls | cut -d, -f1-3
ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV)
```

Лістинг 2 - Метадані файлу та визначення типів файлів

4 Виконав чотири різні пошуки. Пошук за іменем: find /usr/include -maxdepth 1 -name "stdio*.h" | sort (ключ -maxdepth 1 не дозволяє заходити в підкаталоги, -name задає шаблон імені, sort упорядковує результат). Пошук за розміром — критерій свого варіанта: find /usr/include -name "*.h" -size +10k (суфікс k означає кілобайти, а знак + — «більше за», тобто шукаються файли, більші за 10 КБ); спершу для наочності обмежив глибину й вивів файли на літеру s, а потім порахував усі такі файли в усьому каталозі командою wc -l. Пошук за часом зміни: find ~/lab7 -type f -mmin -30 (ключ -type f залишає лише звичайні файли, -mmin -30 — змінені менш ніж 30 хвилин тому). Пошук за вмістом: grep -c "size_t" src/*.h (ключ -c виводить для кожного файлу кількість рядків, що містять заданий текст). Усі результати збіглися з очікуваними: три файли, скопійовані на початку роботи, потрапили і до пошуку за розміром, і до пошуку за часом.

```
nikita@ubuntu-vm:~/lab7$ find /usr/include -maxdepth 1 -name "stdio*.h" | sort
/usr/include/stdio.h
/usr/include/stdio_ext.h

nikita@ubuntu-vm:~/lab7$ find /usr/include -maxdepth 1 -name "s*.h" -size +10k | sort
/usr/include/signal.h
/usr/include/stdio.h
/usr/include/stdlib.h
/usr/include/string.h

nikita@ubuntu-vm:~/lab7$ find /usr/include -name "*.h" -size +10k | wc -l
387

nikita@ubuntu-vm:~/lab7$ find ~/lab7 -type f -mmin -30 | sort
/home/nikita/lab7/src/stdio.h
/home/nikita/lab7/src/stdlib.h
/home/nikita/lab7/src/string.h

nikita@ubuntu-vm:~/lab7$ grep -c "size_t" src/*.h
src/stdio.h:39
src/stdlib.h:31
src/string.h:52
```

Лістинг 3 - Пошук файлів за іменем, розміром, часом зміни та вмістом

5 Створив жорстке й символічне посилання на файл src/string.h: жорстке командою ln src/string.h docs/string-hard.h, символічне — командою ln -s ../src/string.h docs/string-soft.h (шлях символічного посилання записується відносно каталогу, де лежить саме посилання, тому на початку стоїть ../). Команда ls -l docs/ показала, що в жорсткого посилання лічильник імен дорівнює 2 і воно виглядає як

звичайний файл, а символічне починається з літери l і має стрілку на ціль. Командою stat -c "%i %n" порівняв номери inode: у src/string.h і docs/string-hard.h вони однакові (1314874), тобто це два імені одного файлу, а в символічного посилання inode власний (1314902), бо воно є окремим маленьким файлом із текстом шляху. Потім видалив оригінал командою rm src/string.h. Жорстке посилання залишилося робочим: head -n 1 вивела перший рядок файлу, а лічильник імен зменшився до 1. Символічне посилання «повисло»: воно зберігало лише шлях ../src/string.h, за яким тепер нічого немає, тому cat повідомила No such file or directory.

```
nikita@ubuntu-vm:~/lab7$ ln src/string.h docs/string-hard.h
nikita@ubuntu-vm:~/lab7$ ln -s ../src/string.h docs/string-soft.h
nikita@ubuntu-vm:~/lab7$ ls -l docs/
total 20
-rw-r--r-- 2 nikita nikita 17305 Oct 27 14:08 string-hard.h
lrwxrwxrwx 1 nikita nikita    15 Oct 27 14:24 string-soft.h -> ../src/string.h

nikita@ubuntu-vm:~/lab7$ stat -c "%i %n" src/string.h docs/string-hard.h docs/string-
   soft.h
1314874 src/string.h
1314874 docs/string-hard.h
1314902 docs/string-soft.h

nikita@ubuntu-vm:~/lab7$ rm src/string.h
nikita@ubuntu-vm:~/lab7$ ls -l docs/
total 20
-rw-r--r-- 1 nikita nikita 17305 Oct 27 14:08 string-hard.h
lrwxrwxrwx 1 nikita nikita    15 Oct 27 14:24 string-soft.h -> ../src/string.h

nikita@ubuntu-vm:~/lab7$ head -n 1 docs/string-hard.h
/* Copyright (C) 1991-2024 Free Software Foundation, Inc.

nikita@ubuntu-vm:~/lab7$ cat docs/string-soft.h
cat: docs/string-soft.h: No such file or directory
```

Лістинг 4 - Жорстке й символічне посилання, видалення оригіналу

Таблиця 1 – Порівняння жорсткого й символічного посилань за результатами пункту 5

Характеристика	Жорстке посилання (ln)	Символічне посилання (ln -s)
Що зберігає	той самий inode (1314874)	текст шляху ../src/string.h (власний inode 1314902)
Номер inode порівняно з оригіналом	збігається	відрізняється
Розмір у ls -l	17305 байт (розмір самого файлу)	15 байт (довжина шляху)
Лічильник імен у ls -l	2 (після видалення оригіналу — 1)	1

Характеристика	Жорстке посилання (ln)	Символічне посилання (ln -s)
Вигляд у ls -l	звичайний файл -rw-r--r--	lrwxrwxrwx зі стрілкою на ціль
Після видалення оригіналу	дані доступні, head вивела рядок	посилання «висить», cat видала помилку

6 Виконав операції копіювання, переміщення та перейменування в каталозі bak. Звичайне копіювання: cp src/stdio.h bak/stdio-copy.h. Копіювання з ключем -p: cp -p src/stdlib.h bak/stdlib-p.h. Для порівняння вивів ls -l --time-style=long-iso (формат дати з роком і хвилинами). Ключ -p зберіг час зміни оригіналу (14:08), а звичайний cp поставив час створення копії (14:31); крім часу, -p зберігає також власника, групу та права (власника для звичайного користувача зберегти неможливо, а root міг би). Далі створив підкаталог bak/old і перемістив туди копію командою mv bak/stdio-copy.h bak/old/ (переміщення), а потім перейменував її командою mv bak/old/stdio-copy.h bak/old/stdio-v1.h (перейменування). Номер inode перед і після двох операцій не змінився (1314921): у межах однієї файлової системи mv лише змінює запис у каталозі, а не копіює дані, тому і переміщення, і перейменування є однією й тією самою операцією.

```
nikita@ubuntu-vm:~/lab7$ cp src/stdio.h bak/stdio-copy.h
nikita@ubuntu-vm:~/lab7$ cp -p src/stdlib.h bak/stdlib-p.h
nikita@ubuntu-vm:~/lab7$ ls -l --time-style=long-iso src/stdio.h bak/stdio-copy.h
   bak/stdlib-p.h
-rw-r--r-- 1 nikita nikita 29037 2026-10-27 14:31 bak/stdio-copy.h
-rw-r--r-- 1 nikita nikita 33148 2026-10-27 14:08 bak/stdlib-p.h
-rw-r--r-- 1 nikita nikita 29037 2026-10-27 14:08 src/stdio.h

nikita@ubuntu-vm:~/lab7$ stat -c "%i %n" bak/stdio-copy.h
1314921 bak/stdio-copy.h
nikita@ubuntu-vm:~/lab7$ mkdir bak/old
nikita@ubuntu-vm:~/lab7$ mv bak/stdio-copy.h bak/old/
nikita@ubuntu-vm:~/lab7$ mv bak/old/stdio-copy.h bak/old/stdio-v1.h
nikita@ubuntu-vm:~/lab7$ stat -c "%i %n" bak/old/stdio-v1.h
1314921 bak/old/stdio-v1.h
```

Лістинг 5 - Копіювання, переміщення та перейменування, різниця cp і cp -p

7 Змонтував змінний носій у ~/lab7/mnt. Спершу командою lsblk -e7 (ключ -e7 приховує віртуальні loop-пристрої snap) знайшов носій: це /dev/sdb1 обсягом 14,5 ГБ. Ubuntu змонтувала флешку автоматично в /media/nikita/FLASH, тому спершу відмонтував її командою sudo umount /dev/sdb1. Тип файлової системи й UUID визначив командою sudo blkid /dev/sdb1: файлова система vfat, UUID 5E3A-7B21. Оскільки vfat не зберігає власників і прав Unix, змонтував з опціями uid і gid: sudo mount -o uid=$(id -u),gid=$(id -g) /dev/sdb1 ~/lab7/mnt (підстановки $

(id -u) і $(id -g) повертають мої UID і GID, тож власником файлів на носії стає звичайний користувач). Тип файлової системи та вільне місце підтвердила команда df -hT: вільно 14 ГБ. Скопіював на носій файл src/stdio.h без sudo і перевірив його командою ls -l: права rwxr-xr-x задано для всього носія, а не успадковано з оригіналу. Потім відмонтував носій командою sudo umount ~/lab7/mnt. Порожній вивід findmnt і код повернення 1 (echo $?) підтвердили, що точку монтування звільнено і флешку можна безпечно вилучати.

```
nikita@ubuntu-vm:~/lab7$ lsblk -e7
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   30G  0 disk
├─sda1   8:1    0    1M  0 part
└─sda2   8:2    0   30G  0 part /
sdb      8:16   1 14.5G  0 disk
└─sdb1   8:17   1 14.5G  0 part /media/nikita/FLASH
sr0     11:0    1 1024M  0 rom

nikita@ubuntu-vm:~/lab7$ sudo umount /dev/sdb1

nikita@ubuntu-vm:~/lab7$ sudo blkid /dev/sdb1
/dev/sdb1: LABEL="FLASH" UUID="5E3A-7B21" BLOCK_SIZE="512" TYPE="vfat"
   PARTUUID="a3f4c2d1-01"

nikita@ubuntu-vm:~/lab7$ sudo mount -o uid=$(id -u),gid=$(id -g) /dev/sdb1 ~/lab7/mnt

nikita@ubuntu-vm:~/lab7$ df -hT ~/lab7/mnt
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/sdb1      vfat   15G  1.2G   14G   9% /home/nikita/lab7/mnt

nikita@ubuntu-vm:~/lab7$ cp src/stdio.h mnt/ && ls -l mnt/stdio.h
-rwxr-xr-x 1 nikita nikita 29037 Oct 27 14:36 mnt/stdio.h

nikita@ubuntu-vm:~/lab7$ sudo umount ~/lab7/mnt

nikita@ubuntu-vm:~/lab7$ findmnt ~/lab7/mnt
nikita@ubuntu-vm:~/lab7$ echo $?
1
```

Лістинг 6 - Монтування, використання та відмонтування змінного носія

8 Проаналізував дисковий простір каталогу /usr/include. П’ять найбільших підкаталогів знайшов командою du -h --max-depth=1 /usr/include 2>/dev/null | sort -rh | head -n 6 (ключ --max-depth=1 підраховує лише підкаталоги першого рівня, sort -rh сортує за людиночитними розмірами від більшого до меншого, head -n 6 залишає шість рядків: перший є сумою всього каталогу, решта п’ять — шукані підкаталоги; 2>/dev/null приховує повідомлення про відмову в доступі). Найбільший підкаталог — linux із заголовками ядра. Різниця між командами: df -h показує вільні та зайняті **байти** (блоки) файлової системи, а df -i — вільні й зайняті **inode**, тобто записи для файлів. Кожен файл і каталог займає один inode, тому на файловій системі з мільйонами дрібних файлів inode можуть закінчитися раніше за байти: тоді запис неможливий, хоча df -h показує вільне місце. На

кореневому розділі зайнято лише 16% inode при 50% місця, тож цей ризик мені не загрожує.

```
nikita@ubuntu-vm:~/lab7$ du -h --max-depth=1 /usr/include 2>/dev/null | sort -rh | head
   -n 6
14M /usr/include
5.6M    /usr/include/linux
2.4M    /usr/include/x86_64-linux-gnu
1.1M    /usr/include/drm
620K    /usr/include/asm-generic
520K    /usr/include/rdma

nikita@ubuntu-vm:~/lab7$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        30G   14G   14G  50% /

nikita@ubuntu-vm:~/lab7$ df -i /
Filesystem      Inodes  IUsed   IFree IUse% Mounted on
/dev/sda2      1966080 312406 1653674   16% /
```

Лістинг 7 - Аналіз дискового простору, df -h і df -i

9 Створив архів каталогів src і docs у форматі zip: zip -ry bak/lab7-2026-10-27.zip src docs (ключ -r додає каталоги рекурсивно, ключ -y зберігає символічні посилання як посилання, а не слідує за ними: без -y zip намагався б прочитати «висяче» посилання string-soft.h і видав би попередження). Дату в імені можна підставити командою $(date +%F). Переглянув вміст архіву без розпакування командою unzip -l, потім розпакував його в /tmp/lab7-out командою unzip bak/lab7-2026-10-27.zip -d /tmp/lab7-out (ключ -d задає каталог розпакування) і перевірив, що розпаковані файли збігаються з оригіналами (cmp). Символічне посилання потрапило в архів як посилання (linking), і після розпакування воно так само вказує на неіснуючий ../src/string.h. Нарешті, вивів підсумкову структуру каталогів командою tree ~/lab7.

```
nikita@ubuntu-vm:~/lab7$ zip -ry bak/lab7-2026-10-27.zip src docs
  adding: src/ (stored 0%)
  adding: src/stdio.h (deflated 74%)
  adding: src/stdlib.h (deflated 73%)
  adding: docs/ (stored 0%)
  adding: docs/string-hard.h (deflated 70%)
  adding: docs/string-soft.h (stored 0%)

nikita@ubuntu-vm:~/lab7$ ls -l bak/lab7-2026-10-27.zip
-rw-rw-r-- 1 nikita nikita 22552 Oct 27 14:47 bak/lab7-2026-10-27.zip

nikita@ubuntu-vm:~/lab7$ unzip -l bak/lab7-2026-10-27.zip
Archive:  bak/lab7-2026-10-27.zip
  Length      Date    Time    Name
---------  ---------- -----   ----
        0  2026-10-27 14:26   src/
    29037  2026-10-27 14:08   src/stdio.h
    33148  2026-10-27 14:08   src/stdlib.h
        0  2026-10-27 14:24   docs/
    17305  2026-10-27 14:08   docs/string-hard.h
       15  2026-10-27 14:24   docs/string-soft.h
---------                     -------
    79505                     6 files

nikita@ubuntu-vm:~/lab7$ unzip bak/lab7-2026-10-27.zip -d /tmp/lab7-out
```

```
Archive:  bak/lab7-2026-10-27.zip
   creating: /tmp/lab7-out/src/
  inflating: /tmp/lab7-out/src/stdio.h
  inflating: /tmp/lab7-out/src/stdlib.h
   creating: /tmp/lab7-out/docs/
  inflating: /tmp/lab7-out/docs/string-hard.h
   linking: /tmp/lab7-out/docs/string-soft.h  -> ../src/string.h
finishing deferred symbolic links:
  /tmp/lab7-out/docs/string-soft.h -> ../src/string.h

nikita@ubuntu-vm:~/lab7$ cmp src/stdio.h /tmp/lab7-out/src/stdio.h && echo "файли
   ідентичні"
файли ідентичні

nikita@ubuntu-vm:~/lab7$ du -sh src docs
72K src
24K docs

nikita@ubuntu-vm:~/lab7$ tree ~/lab7
/home/nikita/lab7
├── bak
│   ├── lab7-2026-10-27.zip
│   ├── old
│   │   └── stdio-v1.h
│   └── stdlib-p.h
├── docs
│   ├── string-hard.h
│   └── string-soft.h -> ../src/string.h
├── mnt
└── src
    ├── stdio.h
    └── stdlib.h

5 directories, 7 files
```

Лістинг 8 - Створення, перегляд і розпакування архіву zip, підсумкова структура каталогів

Таблиця 2 – Обсяг архіву та вихідних даних

Показник	Значення
Обсяг вихідних даних (вміст файлів, unzip -l)	79 505 байт (≈ 77,6 КБ)
Місце на диску, що займають src і docs (du -sh, блоки по 4 КБ)	96 КБ (72 КБ + 24 КБ)
Обсяг архіву lab7-2026-10-27.zip	22 552 байт (≈ 22 КБ)
Коефіцієнт стиснення (вихідні дані / архів)	79 505 / 22 552 ≈ 3,5 : 1
Частка архіву від вихідних даних	≈ 28% (економія ≈ 72%)

Архів майже у 3,5 раза менший за вихідні дані, бо заголовні файли — це текст із великою кількістю повторюваних слів (#include, size_t, extern), який алгоритм deflate стискає на 70–74%. Файли, що вже стиснені (JPEG, MP4, інші архіви), майже не стиснулися б. Символічне посилання потрапило в архів без стиснення (stored), бо містить лише 15 байт шляху. Різниця між 79 505 байтами

вмісту та 96 КБ на диску пояснюється тим, що du рахує зайняті блоки по 4 КБ разом із блоками самих каталогів, а не байти.

10 Виконав додаткове завдання варіанта №7: пояснив різницю між du і df для того самого каталогу /usr/include. Для цього виконав чотири команди: du -sh /usr/include (сумарний обсяг каталогу в блоках), du -sh --apparent-size /usr/include (сума «видимих» розмірів файлів у байтах), df -h /usr/include (стан файлової системи, до якої належить каталог) і findmnt -T /usr/include (на якому розділі лежить каталог). Результат показав, що du для каталогу дає 14 МБ, а df для того самого шляху — 30 ГБ загального обсягу й 14 ГБ зайнятого, тобто дані всього кореневого розділу /dev/sda2, на якому цей каталог лише один із багатьох.

```
nikita@ubuntu-vm:~$ du -sh /usr/include
14M /usr/include

nikita@ubuntu-vm:~$ du -sh --apparent-size /usr/include
11M /usr/include

nikita@ubuntu-vm:~$ df -h /usr/include
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2        30G   14G   14G  50% /

nikita@ubuntu-vm:~$ findmnt -T /usr/include
TARGET SOURCE    FSTYPE OPTIONS
/      /dev/sda2 ext4   rw,relatime,errors=remount-ro
```

Лістинг 9 - Порівняння du і df для каталогу /usr/include

Різниця між командами полягає у джерелі даних. Команда du сама обходить дерево каталогу, сумує розміри (за замовчуванням — зайняті блоки по 4 КБ, тому показує 14 МБ, а з ключем --apparent-size — реальні розміри файлів у байтах, 11 МБ), і відповідає на питання «скільки займає цей каталог». Команда df не дивиться на файли, а запитує в драйвера файлової системи загальні лічильники її блоків (системний виклик statfs), тому відповідає на питання «скільки заповнено розділ, на якому лежить цей шлях», і для будь-якого каталогу цього розділу показує однакові значення. Тому числа du і df не збігаються й не мають збігатися. Розбіжності між ними можливі навіть тоді, коли за du перевіряється весь розділ: df враховує службові структури (журнал, таблиці inode), резервні блоки (за замовчуванням 5% ext4 зарезервовано для root) і файли, видалені, але ще відкриті якимось процесом, а du бачить лише файли, які ще мають ім’я в дереві; жорстке

посилання du рахує один раз; розріджені файли займають менше місця, ніж їхній видимий розмір.

11 Оформив звіт і зробив висновок.

Відповіді на контрольні питання

Достатній рівень

1. Чому mv виконує і переміщення, і перейменування? Що відбувається у файловій системі в кожному випадку?

Для файлової системи перейменування й переміщення в межах одного розділу — це та сама операція: змінюється запис у каталозі (пара «ім’я — номер inode»), а самі дані та inode лишаються на місці. При перейменуванні змінюється ім’я в тому самому каталозі, при переміщенні запис вилучається зі старого каталогу й додається до нового; саме тому такі операції миттєві, навіть для файлу на 10 ГБ. У цій роботі номер inode (1314921) не змінився ні після mv bak/stdio-copy.h bak/old/, ні після перейменування на stdio-v1.h. Інакше буде, якщо переміщувати файл на іншу файлову систему (наприклад, на флешку): inode там інші, тому mv фактично копіює дані й видаляє оригінал.

2. Запишіть команду, яка знайде в /var/log усі файли більші за 1 МБ, змінені за останню добу.

Команда: sudo find /var/log -type f -size +1M -mtime -1. Умови в find за замовчуванням поєднуються логічним «і»: -type f залишає лише звичайні файли, -size +1M — файли більші за 1 МБ (знак + означає «більше», суфікс M — мегабайти), -mtime -1 — змінені менш ніж 24 години тому (-mtime +1 означало б давніше). sudo потрібен, бо частина журналів у /var/log недоступна звичайному користувачеві; інакше довелося б додати 2>/dev/null, щоб приховати повідомлення про відмову в доступі.

3. Чим жорстке посилання відрізняється від символічного? Наведіть задачу, де потрібне саме символічне.

Жорстке посилання — це ще один запис у каталозі, що вказує на той самий inode: обидва імена рівноправні, а дані зникають лише тоді, коли видалено останнє ім’я. Воно неможливе між різними файловими системами та на каталоги. Символічне посилання — окремий маленький файл із текстом шляху до цілі; воно працює між файловими системами і для каталогів, але ламається, якщо ціль переміщено чи видалено (у моїй роботі string-soft.h після видалення оригіналу «повисло»). Символічне посилання потрібне, наприклад, коли треба перемикати версії програми: каталог /opt/app — символічне посилання на /opt/app-2.1, і при оновленні достатньо перестворити одне посилання на /opt/app-2.2, не змінюючи шляхів у налаштуваннях. Ще один приклад — посилання на файл, що лежить на іншому розділі.

4. Чому після видалення оригіналу жорстке посилання працює, а символічне — ні?

Жорстке посилання вказує не на ім’я, а безпосередньо на inode з даними. Команда rm видаляє лише одне ім’я (запис у каталозі) і зменшує лічильник Links; поки він більший за нуль, файл існує. У моєму випадку лічильник зменшився з 2 до 1, тому head через docs/string-hard.h прочитала файл. Символічне посилання зберігає лише текст шляху ../src/string.h; після видалення оригіналу за цим шляхом нічого немає, тому cat повідомила No such file or directory.

5. Чому в /etc/fstab носії позначають UUID, а не іменем /dev/sdb1?

Імена на кшталт /dev/sdb1 ядро видає за порядком виявлення пристроїв, і він може змінитися: після перезавантаження з підключеною додатковою флешкою чи після зміни порядку дисків /dev/sdb1 може вказувати на зовсім інший носій, а система змонтує не те й не туди. UUID записується у файловій системі під час її створення й супроводжує дані, яким би не було ім’я пристрою (у моєму випадку 5E3A-7B21 для флешки з blkid). Тому запис у /etc/fstab за UUID завжди вказує на потрібний носій.

6. Що робити, якщо umount повідомляє «target is busy»?

Це означає, що якийсь процес використовує файли на цьому носії: відкрив файл або має поточним каталогом. Причину знаходять командою lsof +D /mnt/lab

(або fuser -vm /mnt/lab): вона виводить процеси та файли. Далі треба закрити програму, завершити процес (спершу штатно, а kill — лише за потреби) або вийти з каталогу носія в оболонці (cd ~), а потім повторити umount. Примусове відмонтування (umount -f, umount -l) — крайній засіб, бо можна втратити незаписані дані. Тому носій обов'язково відмонтовують перед вилученням: система тримає дані в кеші й записує їх на диск відкладено.

7. Чому df -h може показувати вільне місце, коли система повідомляє про його брак?

Бо df -h рахує байти (блоки), а запис нового файлу потребує ще й вільного inode — запису про файл у таблиці inode. Якщо на розділі створено дуже багато дрібних файлів, inode можуть закінчитися раніше за байти, і система повідомляє «No space left on device», хоч блоків досить. Це перевіряють командою df -i: у стовпці IUse% буде 100%. Інші причини — квоти користувача, резервні блоки, які ext4 тримає для root (за замовчуванням 5%), та видалені файли, які все ще тримає відкритими процес (їх видно через lsof +L1).

8. Поясніть подвійне розширення .tar.gz і запишіть команду перегляду вмісту такого архіву.

У Linux архівування й стиснення історично розділені: tar збирає дерево файлів в один потік (архів .tar), а gzip стискає цей потік (.gz). Подвійне розширення .tar.gz описує саме ці два послідовні перетворення: файли → tar → gzip. Команда перегляду вмісту без розпакування: tar -tzvf archive.tar.gz, де t — показати вміст (list), z — архів стиснуто gzip, v — показувати докладно, f archive.tar.gz — ім'я архіву, яке має стояти останнім у групі ключів. Для формату варіанта (zip) аналогом є команда unzip -l archive.zip, але zip поєднує архівування й стиснення в одному форматі.

Висновок: За цю лабораторну роботу я на практиці опанував роботу з файловою системою Linux у командному рядку. Навчився створювати структуру каталогів і виконувати повний цикл файлових операцій (mkdir, cp, cp -p, mv, rm, ln) та зрозумів, що переміщення й перейменування в межах однієї файлової системи є лише зміною запису в каталозі. Опанував пошук за іменем, розміром (файли .h більші за 10 КБ у /usr/include), часом зміни та вмістом за допомогою find і grep, прочитав метадані командою stat і визначив реальний тип файлів командою file. Порівняв жорсткі й символічні посилання за номерами inode та поведінкою після видалення оригіналу, змонтував і коректно відмонтував флешку з файловою системою vfat з опціями uid і gid. Проаналізував дисковий простір командами du, df -h і df -i та з'ясував, чим du відрізняється від df для того самого каталогу, створив архів zip і розпакував його, отримавши стиснення близько 3,5 : 1. Єдине дерево каталогів із монтуванням зручніше за літери дисків у Windows: будь-який носій приєднується в довільну точку дерева, шляхи до програм і даних не залежать від того, на якому фізичному пристрої вони лежать, а розширити місце можна, змонтувавши новий диск у потрібний каталог.